

Architectures Parallèles

TP1: x86_64 ASSEMBLY - BASICS

UVSQ - M1 CHPS

The goal of this first lab is to get you familiar with the basics of x86_64 assembly. You will be provided with an archive containing code to complete for each question.

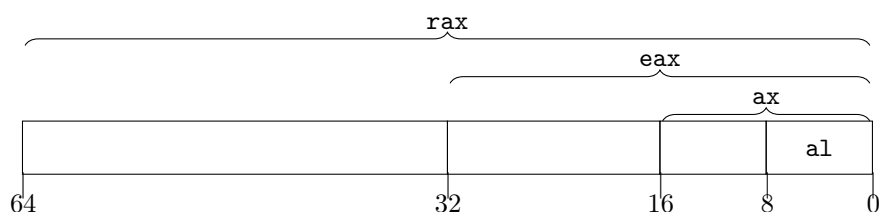
0 Cheatsheet

0.1 Registers

The following table lists the "general purpose" registers (GPRs) of the x86_64 architecture, as well as some special registers:

64 bits	Comment	Volatile	32 bits	16 bits	8 bits
rax	Functions' return values (by convention)	✓	eax	ax	al
rbx			ebx	bx	bl
rcx		✓	ecx	cx	cl
rdx		✓	edx	dx	dl
rsi		✓	esi	si	sil
rdi		✓	edi	di	dil
rbp	Base pointer (base of the stack)		ebp	bp	bpl
rsp	Stack pointer (top of the stack)		esp	sp	spl
r8		✓	r8d	r8w	r8b
r9		✓	r9d	r9w	r9b
r10		✓	r10d	r10w	r10b
r11		✓	r11d	r11w	r11b
r12			r12d	r12w	r12b
r13			r13d	r13w	r13b
r14			r14d	r14w	r14b
r15			r15d	r15w	r15b
rip	Address of the next instruction to be executed	N/A			
rflags	Contains flags set by some instructions	N/A			

Depending on the size of the manipulated types, these registers can be accessed in 64, 32, 16 or 8 bits modes. These variants correspond to the **lower parts** of the register:



Besides having specific functions for some of them (see the "**Comment**" column), GPRs can, by convention, be of two different types:

- **caller-saved**: also called *volatile*, these registers are used to save temporary values that do not need saving between function calls.
- **callee-saved**: also called *non-volatile*, these registers are used to save values that must be preserved over function calls. A function that uses them needs to backup and restore their original values.

0.2 Syntaxes

For historical reasons, there are two syntaxes in x86 assembly: **AT&T** and **Intel**. Here are the differences between them:

- **Parameters order**

AT&T uses the "sources $\rightarrow$ destination" scheme.

Example: "movq \$42, %rax" stores the value 42 in the **rax** register.

Intel uses the "destination $\leftarrow$ sources" scheme.

Example: "mov rax, 42" stores the value 42 in the **rax** register. Note how this instruction would not make sense in AT&T syntax.

- **Instruction suffixes**

AT&T uses suffixes on instruction mnemonics, depending on the size of operands (q for quad-words (64 bits), l for long (32 bits), w for words (16 bits), b for byte (8 bits))

Example: "movl \$42, %eax" stores the value 42 in the 32-bits wide **eax** register (as opposed to the previous example using the 64-bits version of the register).

Intel does not use suffixes on mnemonics.

- **Operands prefixes**

In **AT&T** syntax, immediates values are prefixed with a "\$" and registers are prefixed with a "%".

In **Intel** syntax, there are no prefixes on instruction operands.

- **Addresses computation**

AT&T uses the following syntax: movq (%rbx, %rcx, 8), %rax

Intel uses the following syntax: mov rax, [rbx+rcx*8]

(see the next section for more details on address computation in memory operands)

By convention, this document uses the **Intel** syntax.

0.3 Instructions

The following table lists some basic instructions:

Instruction	Description	C equivalent
add rax, rbx	Integer addition	rax += rbx;
sub rax, rbx	Integer subtraction	rax -= rbx;
imul rax, rbx	Integer multiplication	rax *= rbx;
mov rax, rbx	Copy content of a register into another	rax = rbx;
mov rax, [rbx]	Load value from memory to a register	rax = *rbx;
mov [rax], rbx	Store value from a register into memory	*rax = rbx;
cmp rax, rbx	Compare operands and set the flag register	rax == rbx
je addr	Read the flag register and jump if equal	
jne addr	Read the flag register and jump if non-equal	
jmp addr	Jump unconditionally	
push rax	Push an element on top of the stack	
pop	Pop the element on top of the stack	
call f	Call a function	f(...);
ret	Return from a function	return ...;

A more complete list of x86 instructions can be found at <https://www.felixcloutier.com/x86/>.

Instruction operands can be of the following types:

Operand type	Examples	C equivalent
Immediate	<code>mov rax, 42</code> <code>imul r8, 2</code>	<code>rax = 42;</code> <code>r8 *= 2;</code>
Register	<code>mov rax, rbx</code> <code>add r9, r10</code>	<code>rax = rbx;</code> <code>r9 += r10;</code>
Memory	<code>mov [rbx], r12</code> <code>mov rax, [r11+16]</code>	<code>*rbx = r12;</code> <code>rax = *(r11+16);</code>

Moreover, memory operands can be addressed in multiple ways (non exhaustive list, addresses are computed in bytes):

Addressing form	Example
Base register	<code>mov rax, [rbx]</code>
Base register + displacement	<code>mov rax, [rbx+8]</code>
Base register + Index register	<code>mov rax, [rbx+r8]</code>
Base register + Index register * scaling factor	<code>mov rax, [rbx+r8*8]</code>

Finally, the **address** operand of the jump instructions corresponds to the address of the instruction to jump to. In practice, it is easier to define *labels* in the following way:

```
je mylabel
[...]  
mylabel:  
mov ..., ...
```

If the `rflags` register is set to equal, execution will jump to the `mov` instruction below `mylabel`.

0.4 A practical guide to writing assembly

0.4.1 Inline ASM

One of the simplest ways to write assembly is to use **inline ASM**, which is an extension supported by most modern compilers. It allows writing assembly snippets directly from a high level language (such as C).

When writing inline ASM, up to four sections can be specified (or left empty if not needed):

- the **ASM template section**, containing the assembly code to generate in a string. This is the only mandatory section.
- the **outputs section**, indicating in which C variables the result(s) should be stored.
- the **inputs section**, indicating which C variable should be passed to the ASM snippet.
- the **clobber sections**, to preserve values of registers that are modified by the ASM snippet, and that are use neither as input or output.

These sections are separated by a ":".

Inputs and outputs can have multiple "constraints":

- "**r**" indicates that the operand is neither an input or an output and will be affected to any GPR.
- "**=r**" indicates that the operand is only an output and will be affected to any GPR.
- "**+r**" indicates that the operand is both an output and an input, and will be affected to any GPR.

Example in C:

```
1  int a = 1; int b = 2;
2  asm(
3      "add %0, %1;"
4      : "+r" (a) // Output section (%0 = a). Notice how a is also an input,
5                  // but is listed in the output section.
6      : "r" (b) // Input section (%1 = b)
7      : // Clobber section (empty)
8  );
9  // From there, a == 3
```

Operands can also be given names. This is especially useful when the ASM listings become complex.

Example in C:

```
1      int a = 1;
2      asm(
3          "mov r8, 2;"
4          "add %[foo], r8;"
5          : [foo]"+r" (a) // Output section (%[foo] = a)
6          : // Input section (empty)
7          : "r8" // Clobber section
8      );
9      // From there, a ==3
10     // Also the original value of r8 is preserved,
11     // preventing later issues in the code.
```

By default, the GCC compiler will use AT&T syntax. Intel syntax can be used by adding the `-masm=intel` flag.

0.4.2 Compiling ASM files

It is also possible to compile files entirely written in assembly using GCC:

```
gcc [-masm=intel] -c foo.s
```

... where `foo.S` is the file containing the assembly source. Once again, AT&T syntax is the default. However, this is often more complex than writing inline assembly, because this requires handling the **calling convention**. This is why we are going to use inline ASM in these labs.

0.4.3 Reading compiler generated assembly

If you are stuck on a question for instance, it can be helpful to have a look at the assembly generated by the compiler:

```
gcc [-masm=intel] -S foo.c -o foo.s
```

It is also possible to disassemble a binary using the `objdump` command and the `-d` flag:

```
objdump -d foo
```

If the binary was built with debug info, it is possible to match the assembly with source lines and get an output that is easier to read with the `-S` flag:

```
gcc -g -o foo foo.c
objdump -d [-M intel] -S foo
```

1 Dot product

1.1 2 dimensions dot product

The following code computes the dot product between two vectors $a = (a_x, a_y)$ and $b = (b_x, b_y)$, represented by four signed integer values:

```
#include <stdint.h>

int64_t dotprod_2d(int64_t ax, int64_t ay, int64_t bx, int64_t by) {
    return ax*bx + ay*by;
}
```

Write an equivalent function in ASM or inline ASM.

1.2 Generalization to n dimensions

The following code computes the dot product between two vectors $a = (a_1, \dots, a_n)$ and $b = (b_1, \dots, b_n)$ of size n , represented by two arrays of signed integers:

```
1 int64_t dotprod_nd(size_t n, int64_t a[n], int64_t b[n]) {
2     int64_t sum = 0;
3     for(size_t i=0; i<n; ++i) {
4         sum += a[i] * b[i];
5     }
6
7     return sum;
8 }
```

Write an equivalent function in ASM or inline ASM.

Tip: In assembly, loops are implemented using a conditional jump instruction to a label placed at the top of the loop body.

2 Detecting opposite integer signedness

2.1 Naive approach

The following code returns true if a and b have opposite signs, and false otherwise:

```
1 #include <stdbool.h>
2
3 bool opposite_signs(int64_t a, int64_t b) {
4     if((a>=0 && b<0) || (a<0 && b>=0)) {
5         return true;
6     } else {
7         return false
8     }
9 }
```

Write an equivalent function in ASM or inline ASM.

2.2 [Bonus] Branchless approach

Using ONLY bitwise operators, write an equivalent and branchless function in ASM or inline ASM. This will require instructions that are not given in this document.

3 Searching for the minimum of an array

The following code finds and returns the minimum value of an array of signed integers of size n :

```

1  int64_t array_min(size_t n, int64_t array[n]) {
2      if(n == 0)
3          return 0;
4
5      int64_t min = array[0];
6      for(size_t i=1; i<n; ++i) {
7          if(array[i] < min)
8              min = array[i];
9      }
10
11     return min;
12 }

```

Write an equivalent function in ASM or inline ASM.

4 Fibonacci sequence

The Fibonacci sequence is defined as :

$$\begin{cases} f_0 = 1 \\ f_1 = 1 \\ f_i = f_{i-1} + f_{i-2} \quad \text{if } i \geq 2 \end{cases}$$

The following code computes the n -th element of the Fibonacci sequence using the iterative method:

```

1  uint64_t fibonacci(uint64_t n) {
2      if(n <= 1)
3          return 1;
4
5      uint64_t curr; uint64_t prev1=1; uint64_t prev2=1;
6      for(uint64_t i=1; i<n; ++i) {
7          curr = prev1 + prev2;
8          prev2 = prev1;
9          prev1 = curr;
10     }
11
12     return curr;
13 }

```

Write an equivalent function in ASM or inline ASM.